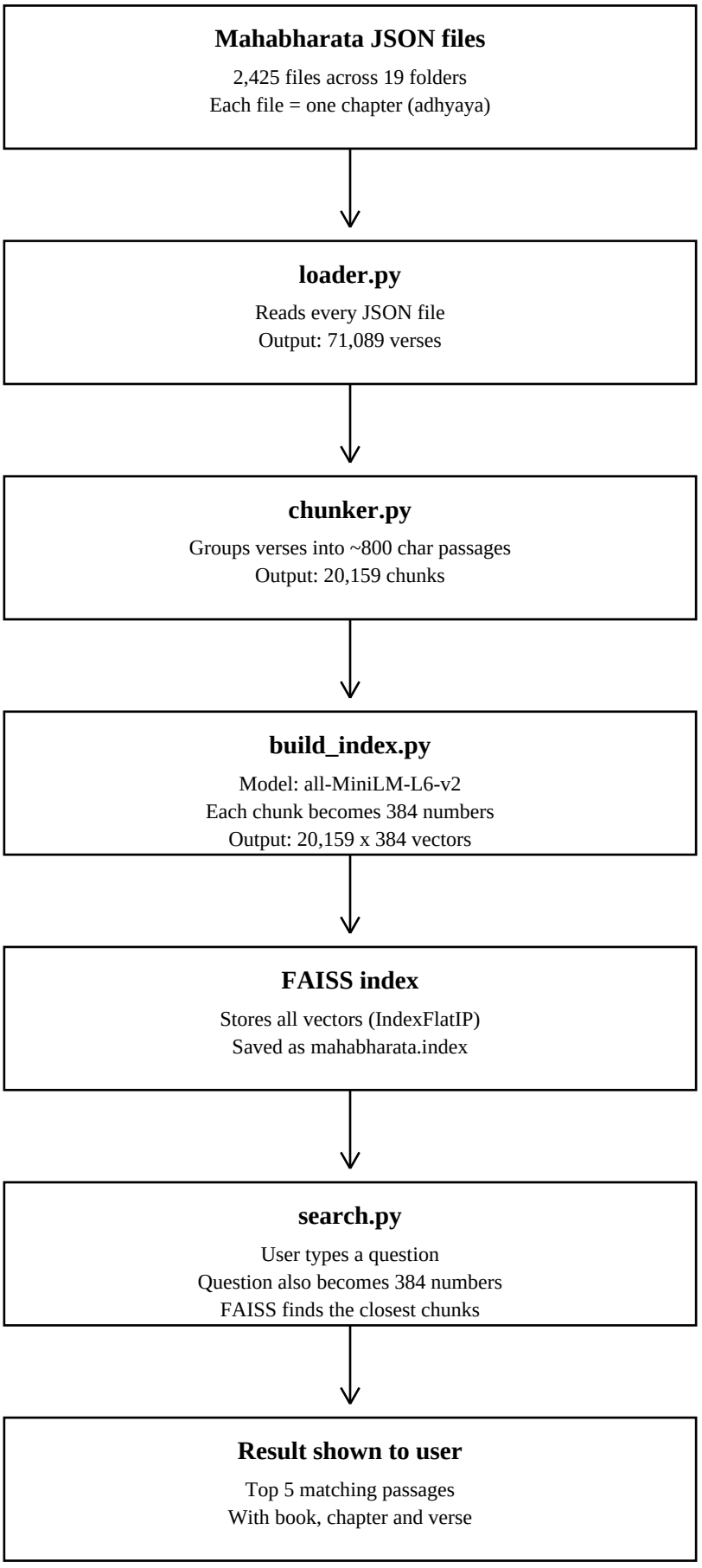
MahabharataGPT - Project Flow

Semantic search over the Mahabharata corpus



Steps 1 to 5 run once. Step 6 runs every time a question is asked.

MahabharataGPT - Concepts Explained

My notes on the ideas behind this project, in simple words

The one-line summary

Instead of making the AI memorise the Mahabharata, I let it look things up. I store the text in a way the computer can search by meaning, fetch the most relevant verses for a question, and then have the AI answer using only those verses. So every answer is grounded in the actual text and can be traced back to a specific verse.

1. What is RAG?

RAG stands for Retrieval-Augmented Generation. It has two halves:

Retrieval means fetching the right passages from my text. **Generation** means the AI writing the answer using those passages.

Think of an open-book exam. The student (the AI) does not memorise the textbook. The student is good at reading and explaining. When a question comes, they find the right page first, read it, and then answer in their own words.

Why not just train the AI on the Mahabharata instead?

Training (fine-tuning) bakes the knowledge into the model itself. That has three problems: it is expensive, you cannot update it easily, and the answers cannot be traced to a source. With RAG, the text stays outside the model. I can change it any time, and every answer points to a real verse.

Why not put the whole book in the prompt?

The Mahabharata is far too long. Even if it fitted, it would cost a lot per question and the model gets less accurate when given very large amounts of text at once. Fetching only the 5 relevant passages is cheaper and more precise.

2. What is semantic search?

Normal search matches words. If you search for "gambling", it only finds verses containing the word "gambling".

Semantic search matches **meaning**. So a question about "gambling" can also find a verse about "a game of dice", even though not a single word matches.

I saw this working in my own project. I asked "who is Yudhishthira?" and it returned verses about his birth from Dharma and his righteousness. It understood I was asking for a description of the person, not just any line with his name in it.

3. What are embeddings?

An embedding turns a piece of text into a list of numbers that represents its meaning. In my project each passage becomes a list of **384 numbers**.

The useful part: passages with similar meaning get similar numbers. So meaning becomes a **location**. You can imagine every passage as a point on a huge map, where related passages sit close together.

An example

Take these three sentences:

1. "Yudhishthira wagered and lost his kingdom in the game of dice."
2. "The Pandava king bet everything on a dice match and was ruined."

3. "Bhima cooked a great feast in the palace kitchen."

Sentences 1 and 2 share almost no words, but they mean nearly the same thing, so their 384 numbers come out close together. Sentence 3 is about something else entirely, so it sits far away.

When a question arrives, it is converted into 384 numbers the same way. Finding a relevant passage then becomes simple geometry: find the nearest points.

Why 384?
That is the output size of the model I used, all-MiniLM-L6-v2. The "L6" means it has 6 transformer layers, which keeps it small and fast. Bigger models produce 768 or more numbers and capture more nuance, but cost more memory and time. 384 is a deliberate trade-off: small, fast, and good enough.

4. What is chunking?

A single verse is often too short to be useful on its own. One verse might just say "But noble Kulapati Shaunaka is now engaged in the room of the holy fire" - meaningless without context.

So I join consecutive verses together into passages of about 800 characters. That is roughly 3 to 4 verses each.

Two rules I follow. First, never mix verses from different chapters, so the citation stays honest. Second, stop once the passage crosses 800 characters.

The trade-off matters: chunks that are too small lose context, and chunks that are too large dilute the meaning and pull in irrelevant text.

5. What is FAISS?

FAISS stands for **Facebook AI Similarity Search**. It was built by Facebook AI Research (now Meta AI). It is pronounced "face".

It is a library that stores a large number of vectors and very quickly finds the ones closest to a given vector. In my project it holds all 20,159 passage vectors and finds the closest matches to a question in milliseconds.

One thing to understand
FAISS only stores numbers. It has no idea what the text says. When I search, it returns **positions** - for example "match number 4,201". I then use that position to look up the actual text in my chunks.json file. That is why I save two files, not one.

Exact vs approximate search
I used IndexFlatIP, which checks every vector and gives perfect results. With only 20,000 vectors this is fast enough. For millions of vectors you would switch to an approximate index like IVF or HNSW, which is much faster but occasionally misses the true best match. That is the trade-off: accuracy against speed.

Why I normalise the vectors
Normalising scales every vector to the same length. Once that is done, the inner product FAISS calculates is the same as cosine similarity, which is the standard way of comparing meaning. It compares the **direction** of the vectors, not their size.

6. Is FAISS a database?

Not exactly, and this is worth being careful about. FAISS is a **library** that lives in memory and gets saved to a file. It is not a database server like MySQL or MongoDB.

People loosely call it a "vector database". True vector databases such as Pinecone, Weaviate, Chroma or pgvector add things on top: proper storage, filtering, and scaling across machines.

My project has no traditional database at all. The source text is just JSON files on disk. For a read-only search system, that is all I need.

7. What is the LLM used for?

The LLM (Claude) is the part that actually writes the answer. Everything before it - chunking, embeddings, FAISS - is machinery for finding the right passages. But finding passages is not the same as answering a question.

Important point: **the LLM does not answer from its own memory.** I feed it the verses I retrieved and instruct it to answer using only those. So its job is reading and writing, not recall.

The division of labour

> *Retrieval decides WHAT to say. The LLM decides HOW to say it.*

FAISS finds the relevant text. The LLM turns scattered verses into one clear, cited answer. It also knows when to say "this is not in the passages" instead of inventing a verse. That instruction is the main protection against hallucination.

8. What is LangChain?

LangChain is a toolkit that connects the steps of an AI app together, so you do not have to wire everything by hand.

In a RAG system there is a small assembly line: take the question, find the passages, put them into a prompt, send it to the LLM, get the answer. LangChain gives ready-made pieces for each step and a clean way to chain them in order. That is where the name comes from.

An analogy

Think of cooking. You could buy raw ingredients and do every step yourself - chop, measure, mix. That works, but it is a lot of manual effort. LangChain is like a meal-kit with pre-portioned components and a recipe card. You get the same meal without rebuilding every basic piece.

The trade-off

LangChain saves boilerplate and gives standard building blocks. But it adds a layer of indirection and extra dependencies. For a small project you can do it directly and keep full control, which is easier to debug and easier to explain.

9. What is FastAPI?

FastAPI turns my code into a web service, so other apps or a browser can use it over the internet. Without it, my project only runs in a terminal on my own laptop.

In simple terms, it lets me say: when a request arrives at this address, run this Python function.

How a request flows through it

1. The user sends a question to the address /ask
2. Uvicorn, the server, receives it
3. FastAPI sees the address and calls the matching function
4. Pydantic checks the incoming data is the right shape
5. My function retrieves passages and calls the LLM
6. The result is converted to JSON and sent back

Things FastAPI gives you free

Automatic checking. I declare that the question must be text. If someone sends something wrong, FastAPI rejects it with a clear error. I write no checking code myself.

Automatic documentation. Open /docs in a browser and there is a ready test page where you can try the endpoint by clicking. This comes purely from the type hints in my code.

Async support. Each request spends most of its time waiting for the LLM to reply. Async lets the server handle other requests during that wait.

One design detail

The embedding model and the FAISS index are loaded **once when the server starts**, not on every request. Loading them each time would add several seconds to every question.

10. FastAPI, Flask or Django?

Django is the full package. It includes a database system, an admin panel, user login, and HTML page templates. You use it for complete websites.

Flask is minimal. It gives you routing and nothing else. You add whatever you need yourself.

FastAPI is the modern middle ground, built specifically for APIs. Light like Flask, but with automatic checking, automatic documentation, and async support.

Why I chose FastAPI

My project is not a website with pages, logins and a big database. It is one endpoint that takes a question and returns an answer. Django's main strengths would go completely unused, so it would be over-engineering. FastAPI gives exactly what is needed and nothing more.

Quick revision - the numbers

JSON files	2,425
Parva folders	19
Verses loaded	71,089
Chunks created	20,159
Chunk size	about 800 characters
Embedding model	all-MiniLM-L6-v2
Embedding dimensions	384
Model layers	6
FAISS index type	IndexFlatIP (exact)
Index build time	about 45 minutes on CPU